

ROADMAP

Junior Developer + Cybersecurity

Luglio 2026 → Giugno 2027

OBIETTIVO

**Arrivare alla primavera/estate 2027 con competenze dimostrabili,
un GitHub solido e progetti reali per candidarsi a ruoli Junior.**

Strategia: 40% studio • 60% pratica • ogni argomento importante produce qualcosa su GitHub

8-10 h/settimana

ritmo sostenibile con lavoro full-time

5-7 progetti

portfolio concreto, non solo certificati

Aprile 2027

inizio candidature selettive

Percorso basato sul syllabus “The Complete Web Development Bootcamp” (Angela Yu / App Brewery), integrato con pratica, portfolio, Linux, networking e cybersecurity.

1. La strategia

Un percorso unico all'inizio, poi una specializzazione più netta nel 2027.

PRINCIPIO GUIDA

Fino a fine 2026: 80% Development / 20% Cybersecurity. Da febbraio 2027: circa 65% Development / 35% Cyber, poi scelta della specializzazione.

- Non limitarsi ai video: ogni concetto importante deve diventare esercizio, feature o mini-progetto.
- Costruire pochi progetti seri e progressivi, con commit puliti e README professionali.
- Usare l'esperienza IT reale come vantaggio: ticketing, asset, utenti, MFA, supporto enterprise.
- Da aprile/maggio 2027 iniziare candidature selettive, senza aspettare di sentirsi "100% pronti".

2. Roadmap annuale

Periodo	Focus	Progetto GitHub	Output
Lug-Ago 2026	HTML, CSS, Bootstrap, JavaScript base	Developer Portfolio	Fondamenta Web + portfolio v1
Set 2026	JavaScript + DOM	Mini Ticket Manager	JS pratico e autonomia
Ott 2026	Git/GitHub + HTTP + API	API Dashboard	REST/API + workflow Git
Nov 2026	React	IT Asset Dashboard	Frontend React
Dic 2026-Jan 2027	Node, Express, PostgreSQL	HelpDesk Pro	Prima vera app Full Stack
Feb 2027	Auth + Security + Deployment	Secure Auth Lab	Ponte Dev → Cyber
Mar 2027	Linux + Networking + Cyber	Cybersecurity Home Lab	Security foundations
Apr 2027	OWASP + Web Security	Web Security Lab	Secure coding / AppSec
Mag 2027	Capstone	SecureDesk	Progetto flagship
Giu 2027	CV, LinkedIn, GitHub, colloqui	Portfolio finale	Candidature attive

3. Settimana tipo - circa 9 ore

Ritmo realistico da mantenere mentre lavori full-time.

Giorno	Tempo	Attività
Lunedì	1h	Corso Udemy + appunti
Martedì	1h30	Corso + esercizi senza copiare
Mercoledì	-	Riposo / vita personale
Giovedì	1h30	Pratica autonoma + Git
Venerdì	45 min	Ripasso leggero / debugging
Sabato	3h	Progetto GitHub
Domenica	1h15	Cyber/Linux + pianificazione

SETTIMANA MINIMA

Settimana normale: ~9h. Settimana impegnativa: ~5h. Settimana terribile: 2h (1h corso + 1h codice). Mai cercare di recuperare con maratone: conta la continuità.

4. Come dividere le ore

- Corso strutturato - circa 3h/settimana: 20-30 minuti di video, poi prova autonoma.
- Pratica autonoma - circa 2h/settimana: rifare senza tutorial ciò che hai appena imparato.
- Progetto GitHub - circa 3h/settimana: una feature piccola, chiara e completabile.
- Cybersecurity - inizialmente ~1h/settimana: Linux, TCP/IP, DNS, HTTP/HTTPS, porte, SSH, permessi.

REGOLA ANTI-TUTORIAL HELL

30 minuti bloccato → cerca aiuto → capisci la soluzione → riscrivila da solo. L'obiettivo è imparare a ragionare, non dimostrare di poter soffrire quattro ore su una parentesi.

5. Luglio-Agosto 2026 - Fondamenta + Portfolio

- Studio: HTML5, CSS3, responsive design, Bootstrap, JavaScript foundations.
- Ore: 3h Udemy • 2h esercizi • 2h portfolio • 1h Git/Linux.

PROGETTO #1 - developer-portfolio

Sito personale professionale: About Me, IT Experience, Technical Skills, Projects, Learning/Certifications e Contact. Prima versione in HTML/CSS/JS; più avanti verrà evoluta in React.

- Output entro fine agosto: portfolio v1 online, GitHub ordinato, basi JavaScript consolidate.
- Obiettivo qualitativo: il sito deve raccontare la transizione IT → Development/Cybersecurity.

6. Settembre 2026 - JavaScript + DOM

- Studio: variables, arrays, objects, functions, loops, DOM, events, async/await, fetch, JSON, error handling.
- Ore: 3h Udemy • 2h esercizi JS • 3h progetto • 1h Git/Linux.

PROGETTO #2 - mini-ticket-manager

Applicazione Help Desk in JavaScript: creare ticket, priorità, stato, categoria, filtri, ricerca, eliminazione e LocalStorage. Idea direttamente collegata alla tua esperienza reale con ServiceNow/Zendesk.

- Output: app funzionante scritta principalmente da te, non copiata da un tutorial.

7. Ottobre 2026 - Git + API + HTTP

- Studio: Git clone/add/commit/push/pull, branch/merge, .gitignore, README, HTTP, REST, JSON e API.
- Ore: 2h30 Udemy • 2h esercizi API • 3h30 progetto • 1h Linux/networking.

PROGETTO #3 - api-dashboard

Dashboard che integra API pubbliche (es. meteo, cambio valuta, GitHub). Focus su fetch, async/await, loading state, error handling e lettura della documentazione API.

- Commit consigliati: feat:, fix:, docs:, refactor: con messaggi semplici e significativi.

8. Novembre 2026 - React

- Studio: Components, Props, JSX, State, Hooks, Events, Forms, Filtering, Search, Routing.
- Ore: 3h React • 2h esercizi • 4h progetto • 1h Cyber/Linux.

PROGETTO #4 - it-asset-dashboard

Dashboard React per la gestione dispositivi aziendali: hostname, seriale, modello, utente, reparto, stato, data acquisto, OS. Ricerca, filtri e viste riepilogative.

- Valore portfolio: progetto credibile perché nasce dalla tua esperienza con il lifecycle dei dispositivi IT.
- Non correre: capire davvero useState, props e flusso dati vale più di finire una sezione in fretta.

9. Dicembre 2026-Gennaio 2027 - Full Stack

- Studio: Node.js, Express, REST, PostgreSQL, CRUD, relazioni dati, middleware.
- Ore: 3h Udemy • 2h backend • 4h HelpDesk Pro • 1h Linux/networking.

PROGETTO #5 - HelpDesk Pro

La versione evoluta del Mini Ticket Manager: React → REST API → Node/Express → PostgreSQL. Utenti, ruoli, ticket, commenti, categorie, asset, priorità e dashboard.

- API di base: GET /tickets, GET /tickets/:id, POST /tickets, PATCH /tickets/:id, DELETE /tickets/:id.
- Output: prima applicazione Full Stack realmente presentabile a un colloquio Junior.

10. Febbraio 2027 - Development incontra Cyber

- Rapporto: circa 65% Development / 35% Cyber.
- Studio: password hashing, bcrypt, sessioni, cookie, JWT concepts, OAuth, environment variables, RBAC.

PROGETTO #6 - secure-auth-lab

Mini-app dedicata ad autenticazione e autorizzazione: login, ruoli Admin/Technician/User, permessi, password sicure e security logs.

- Parallelamente: HelpDesk Pro v2 con autenticazione, ruoli e sicurezza di base.

11. Marzo 2027 - Cybersecurity Fundamentals

- Studio: Linux, TCP/IP, DNS, DHCP, HTTP/HTTPS, porte, firewall, SSH, permessi, processi, servizi, log.
- Ore: 2h Linux/networking • 2h teoria cyber • 2h lab • 3h portfolio/lab • 1h dev maintenance.

PROGETTO #7 - cybersecurity-home-lab

Repository documentale con laboratori: networking, Linux, Wireshark, Nmap, log analysis, web security e incident response. Ogni lab documenta obiettivo, ambiente, passaggi, risultati e cosa hai imparato.

12. Aprile 2027 - OWASP + Web Security

- Studio: OWASP Top 10, SQL Injection, XSS, CSRF, Broken Authentication, Access Control, Security Misconfiguration.

PROGETTO #8 - web-security-lab

Repository educativo su OWASP e secure coding: input validation, password storage, environment variables, query parametrizzate e access control. Usare solo ambienti autorizzati/laboratori.

- Collegamento forte con HelpDesk Pro: documentare come una vulnerabilità può essere prevenuta nel codice.
- Da questo mese: prime candidature selettive se hai JavaScript, React, Git, API, Node/Express basics, SQL e almeno 3 progetti buoni.

13. Maggio-Giugno 2027 - Capstone + Candidature

PROGETTO FLAGSHIP - SecureDesk

Help Desk IT orientato alla sicurezza: React + Node + Express + PostgreSQL, autenticazione, RBAC, ticket, asset, security incident reporting e audit logs. Categorie security: phishing, malware, suspicious login, lost device, data exposure.

- Maggio: 20% studio / 80% costruzione e rifinitura.
- Giugno: CV Developer e CV Cyber separati, LinkedIn aggiornato, GitHub pulito, portfolio online, candidature attive.
- Obiettivo: raccontare una storia coerente - IT Support → Software Development → Security → Secure Application Development.

14. Come deve apparire il GitHub finale

Repository	Cosa dimostra	Priorità
SecureDesk	Capstone Full Stack + sicurezza	★★★★★
HelpDesk Pro	React + Node/Express + PostgreSQL + REST	★★★★★
IT Asset Dashboard	React applicato a un caso IT reale	★★★★☆
Cybersecurity Home Lab	Pratica e documentazione Cyber	★★★★☆
Web Security Lab	OWASP + secure coding	★★★★☆
API Dashboard	HTTP/API/async JavaScript	★★★★☆
Developer Portfolio	Presentazione professionale e deploy	★★★★☆

README MINIMO PER OGNI PROGETTO

About • Features • Technologies • Screenshots • Installation • What I learned • Future improvements. Un recruiter deve capire il progetto senza doverlo installare.

15. Workflow Git consigliato

- Una feature piccola per sessione: es. filtro priorità, login, endpoint, validazione.
- Commit frequenti e leggibili: feat:, fix:, docs:, refactor:.
- Mai caricare secrets, password o chiavi API: usare .env e .gitignore.
- Tenere 5-7 repository seri invece di 40 progetti incompleti.

ESEMPIO DI COMMIT

feat: add ticket priority filtering
 fix: handle API errors
 docs: update README
 refactor: simplify ticket service

16. Scelta della specializzazione - primavera 2027

Junior Developer	Junior Cybersecurity	Evoluzione futura
JavaScript → TypeScript → React → Node → PostgreSQL → Testing → Docker basics	Networking → Linux → Security fundamentals → SIEM → Log analysis → SOC labs → Certificazione	Application Security / DevSecOps: unire sviluppo, sistemi e sicurezza

TRAGUARDO

Con una media di 8-9 ore a settimana per circa 45 settimane: ~360-400 ore di formazione e pratica. Il vero vantaggio competitivo sarà la combinazione tra 5+ anni di esperienza IT, competenze moderne e portfolio concreto.

17. Checklist mensile

- ☐ Ho studiato almeno 70-80% delle ore pianificate?
- ☐ Ho scritto codice senza seguire passo-passo un tutorial?
- ☐ Ho fatto almeno 1-3 commit significativi?
- ☐ Ho migliorato un progetto esistente o completato una feature?
- ☐ So spiegare a voce ciò che ho costruito?
- ☐ Ho aggiornato README/screenshot/documentazione?
- ☐ Ho annotato cosa non ho capito per riprenderlo la settimana successiva?

18. La regola finale

OGNI ARGOMENTO DEVE DIVENTARE QUALCOSA DI VISIBILE

React → un componente reale. API → un'integrazione reale. PostgreSQL → un database reale. Autenticazione → una feature sicura. Linux/Cyber → un lab documentato. Così lo studio si trasforma, settimana dopo settimana, in prove concrete delle tue competenze.

OBIETTIVO: PRIMAVERA/ESTATE 2027 - PRONTO A CANDIDARTI 🚀